Wheels and GitHub Actions

A tutorial on how nabia's C++ library becomes `pip install`-able, everywhere

nabia documentation

October 2026

Abstract

nabia's numerical library is C++; its users work in Python. The bridge between the two is a *wheel*: a pre-compiled Python package. This tutorial explains, from the ground up and with nabia's own files as the running example, what a wheel is and how pip chooses and installs one, how a wheel is built from a CMake project, how cibuildwheel builds one per Python version and platform, and how GitHub Actions runs all of it on Linux, macOS and Windows machines. The last part shows how to write new jobs, how to run, monitor and debug workflows, and how to publish. The files shown are *extracted from the repository when this document is built*, so they are always the ones in use.

Contents

1 The problem, and the map **3**

I Wheels **3**

2 Anatomy of a wheel **3**
2.1 The filename is the compatibility contract 4
2.2 How pip chooses 4

3 Building one wheel **4**
3.1 The build interface: `[build-system]` 4
3.2 What the package is: `[project]` 5
3.3 How CMake is driven: `[tool.scikit-build]` 5
3.4 What goes into the wheel: CMake install rules 6
3.5 Try it 7

4 Installing and using a wheel **7**

II Many wheels **8**

5 All Python versions on one machine **8**

6 cibuildwheel **9**
6.1 Configuration 9
6.2 Running it yourself 10

III GitHub Actions **12**

7 Concepts **12**
7.1 YAML and expressions 12

8 nabia's workflow, piece by piece **13**
8.1 Name and triggers 13
8.2 The build matrix 13
8.3 Collecting everything 14
8.4 Publishing a release 14
8.5 Testing the Windows scripts 15

IV Doing it yourself **15**

9 Writing a job **15**
9.1 Example: the C++ tests on Linux and macOS 16
9.2 Example: the Python project's tests against the Linux wheel 16
9.3 Example: publishing to PyPI without a password 16
9.4 Useful extras 17

10 Running, watching and debugging **17**
10.1 From the terminal 17
10.2 On the web 17
10.3 Reading a failure 17

11 Releasing **18**

12 Costs and limits **18**

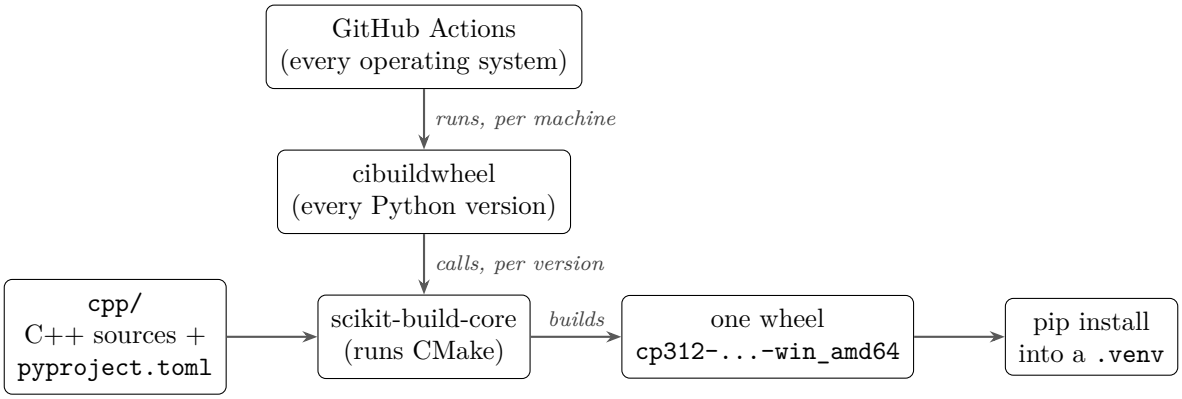
A Glossary **19**

1 The problem, and the map

A Python package written in Python runs anywhere Python runs. `nabia_python_bridge` is not: its heart is `_core`, a shared library compiled from C++ (a `.so` on Linux and macOS, a `.pyd` on Windows). Compiled code is tied to

- an **operating system** (Linux, macOS, Windows) and its C/C++ runtime;
- a **processor architecture** (x86_64 for Intel and AMD, arm64/aarch64 for Apple silicon and ARM servers);
- a **Python version**: an extension module is compiled against the C API of one CPython release (3.12, 3.13, . . .), whose binary interface (ABI) changes between them.

So "making nabia installable" means producing one binary package per combination, testing each, and putting them where pip can find them. Here is how the pieces fit:



File	Role
`cpp/pyproject.toml`	what the package is and how to build it (scikit-build-core) and how to build all variants (cibuildwheel)
`cpp/bindings/CMakeLists.txt`	the extension modules and what goes into the wheel
`cpp/scripts/build_wheels.sh, .ps1`	all wheels of *this* machine (macOS/Linux, Windows)
`.github/workflows/wheels.yml`	all wheels of *all* machines, in GitHub's cloud
`python/scripts/setup_env.sh, .ps1`	installs a wheel into the Python project

Part I
Wheels

2 Anatomy of a wheel

A wheel is a **zip file** with the extension `.whl`. Look inside one:

```
$ python -m zipfile -l dist/nabia_python_bridge-0.1.0-cp314-cp314-macosx_13_0_arm64.whl
nabia_python_bridge/__init__.py
nabia_python_bridge/frames.py                          ... and the other .py files
nabia_python_bridge/_core.cpython-314-darwin.so        the whole C++ library, compiled
nabia_python_bridge/_rb_backtest.cpython-314-darwin.so
nabia_python_bridge-0.1.0.dist-info/METADATA           name, version, dependencies, README
nabia_python_bridge-0.1.0.dist-info/WHEEL              wheel format and tags
nabia_python_bridge-0.1.0.dist-info/RECORD             every file with its SHA-256 hash and size
```

- The files are laid out *exactly as they will be installed*: installing is unzipping into `site-packages`. Nothing is compiled at install time.

- `METADATA` lists the dependencies (`Requires-Dist: numpy>=1.24`), which pip installs too.
- `WHEEL` says `Root-Is-Purelib: false` (it contains compiled code) and repeats the tag.
- `RECORD` lets pip verify the files and uninstall them later.

2.1 The filename is the compatibility contract

`nabia_python_bridge-0.1.0-cp314-cp314-macosx_13_0_arm64.whl`

Part	Name	Meaning
`nabia_python_bridge`	distribution	the package name (`project.name`)
`0.1.0`	version	`project.version`
`cp314`	Python tag	which Python implementation and version can run it: CPython 3.14
`cp314`	ABI tag	which binary interface the compiled code uses. `abi3` would mean the stable ABI (one wheel for many versions); pybind11 modules use the full API, so it is the version itself
`macosx_13_0_arm64`	platform tag	operating system, its minimum version, and the architecture

Platform tags you will meet:

`manylinux_2_28_x86_64`, `_aarch64` Linux with glibc 2.28 or newer (Debian 10+, Ubuntu 20.04+, RHEL 8+). "Many" Linux distributions, because the wheel only uses libraries every one of them has, in versions at least that old. It is built inside an old Linux container on purpose: code compiled against an old glibc runs on newer ones, not the reverse.

`macosx_13_0_arm64`, `_x86_64` macOS 13 or newer, on Apple silicon or Intel. The number comes from `MACOSX_DEPLOYMENT_TARGET` at build time; it tells the compiler which macOS APIs it may use.

`win_amd64`, `win_arm64` 64-bit Windows on Intel/AMD, or on ARM.

`any` no compiled code. The Python projects `nabia` and `nabia_webui` would produce `py3-none-any` wheels: one file for every platform.

2.2 How pip chooses

pip computes the list of tags the running interpreter supports, most specific first, and picks the first wheel that matches. See the list with `python -m pip debug --verbose`. If no wheel matches, pip falls back to the **sdist** (source distribution, a `.tar.gz` of the sources) and compiles it, which needs CMake and a C++20 compiler on that machine. That is why binary packages publish one wheel per platform and Python version: users then never compile.

3 Building one wheel

3.1 The build interface: `[build-system]`

pip does not know CMake. Since PEP 517, a project declares a *build backend* that pip calls in a fresh, isolated environment:

File 1: `cpp/pyproject.toml`: which backend builds the wheel

```
[build-system]
requires = ["scikit-build-core>=0.10", "pybind11>=2.13"]
build-backend = "scikit_build_core.build"
```

1. pip (or `python -m build`, or `uv build`) creates a temporary virtual environment and installs `requires` into it: scikit-build-core and pybind11.
2. It calls the backend's `build_wheel()`.
3. scikit-build-core configures and builds the CMake project, runs the CMake install step into a staging folder, and zips that folder into a wheel with its `dist-info`.

3.2 What the package is: `[project]`

File 2: `cpp/pyproject.toml`: package metadata (becomes `METADATA`)

```
[project]
name = "nabia_python_bridge"
version = "0.1.0"
description = "Python bindings of the nabia C++ numerical library"
readme = "bindings/README.md"
requires-python = ">=3.10"
dependencies = ["numpy>=1.24"]
```

`dependencies` are installed by pip with the wheel. Optional groups are installed only on request, for example with `pip install "nabia_python_bridge[test]"`:

File 3: `cpp/pyproject.toml`: optional dependency groups

```
[project.optional-dependencies]
pandas = ["pandas>=2.0"]
test = ["pytest>=7", "pandas>=2.0"]
examples = ["pandas>=2.0", "matplotlib>=3.7", "yfinance>=0.2"]
```

3.3 How CMake is driven: `[tool.scikit-build]`

File 4: `cpp/pyproject.toml`: scikit-build-core settings

```
[tool.scikit-build]
sdist.exclude = ["build/", "cmake-build-*/", ".idea/", "doc/", "compile_commands.json", "bindings/
    ↪ nabia_python_bridge/_*.so"]
# This C++ project (cpp/) is the CMake source; only the Python bindings are built.
cmake.source-dir = "."
cmake.build-type = "Release"
# CMake installs the whole package (bindings/nabia_python_bridge/*.py, _core and _rb_backtest), so a
# development build's modules or __pycache__ left in the source tree never end up in the wheel.
wheel.packages = []
build-dir = "build/{wheel_tag}"
```

- `cmake.source-dir` is the folder with the top `CMakeLists.txt`: `cpp/` itself.
- `wheel.packages = []`: scikit-build-core copies no Python folder by itself; CMake installs everything (below). This keeps development leftovers out of the wheel.
- `build-dir` keeps a build folder per wheel tag, so rebuilding is incremental.
- `sdist.exclude` keeps build folders and documentation out of the source package.

File 5: `cpp/pyproject.toml`: CMake options for wheel builds

```
[tool.scikit-build.cmake.define]
NABIA_BUILD_PYTHON_BRIDGE = "ON"
NABIA_BUILD_APPS = "OFF"
NABIA_BUILD_TESTS = "OFF"
NABIA_BUILD_EXAMPLES = "OFF"
```

```
NABIA_BUILD_DOCS = "OFF"
# nabia, iir1 and rb_backtest_core are static libraries linked into the extension modules.
CMAKE_POSITION_INDEPENDENT_CODE = "ON"
```

These are ordinary CMake cache variables, as with `-D` on the command line: only the bindings are built (no tests, apps, examples or PDFs), and static libraries are compiled as position-independent code so they can be linked into a shared module on Linux.

3.4 What goes into the wheel: CMake install rules

File 6: `cpp/bindings/CMakeLists.txt`: the modules and the install rules

```
# Python bindings: the C++ side of the nabia_python_bridge wheel (pybind11).
#
#   nabia_python_bridge._core          the whole library (stats, signal, forecast, optimization, risk,
#                                      strategy, regression)
#   nabia_python_bridge._rb_backtest   JSON-configured risk budgeting backtests (apps/rb_backtest),
#                                      used by nabia.portfolio in the Python project
#
# Wheel build (pyproject.toml of this C++ project, scikit-build-core): both modules and the package's
# .py files are installed into the wheel. Development build (CLion, preset python-bridge): the modules
# are written next to the .py files in bindings/nabia_python_bridge/, so the package can be imported
# with PYTHONPATH=cpp/bindings, e.g. to debug the C++ side from a Python script.

find_package(Python 3.10 REQUIRED COMPONENTS Interpreter Development.Module)

# pybind11: from the Python environment (pip install pybind11), else fetched.
execute_process(
    COMMAND "${Python_EXECUTABLE}" -m pybind11 --cmakedir
    OUTPUT_VARIABLE pybind11_cmake_dir
    OUTPUT_STRIP_TRAILING_WHITESPACE
    ERROR_QUIET)
find_package(pybind11 2.13 CONFIG QUIET HINTS "${pybind11_cmake_dir}")
if(NOT pybind11_FOUND)
    message(STATUS "pybind11 not found, fetching it")
    include(FetchContent)
    FetchContent_Declare(pybind11
        GIT_REPOSITORY https://github.com/pybind/pybind11.git
        GIT_TAG        v3.0.1
        GIT_SHALLOW    TRUE
        SYSTEM)
    FetchContent_MakeAvailable(pybind11)
endif()

pybind11_add_module(_core
    src/module.cpp
    src/bind_stats.cpp
    src/bind_signal.cpp
    src/bind_forecast.cpp
    src/bind_optimization.cpp
    src/bind_risk.cpp
    src/bind_strategy.cpp
    src/bind_regression.cpp)
target_link_libraries(_core PRIVATE nabia::nabia)
nabia_set_warnings(_core)

pybind11_add_module(_rb_backtest src/rb_backtest_module.cpp)
target_link_libraries(_rb_backtest PRIVATE rb_backtest_core)
nabia_set_warnings(_rb_backtest)

if(SKBUILD)
    install(TARGETS _core _rb_backtest LIBRARY DESTINATION nabia_python_bridge)
    install(DIRECTORY nabia_python_bridge/ DESTINATION nabia_python_bridge
        FILES_MATCHING PATTERN "*.py" PATTERN "__pycache__" EXCLUDE)
else()
    set_target_properties(_core _rb_backtest PROPERTIES
        LIBRARY_OUTPUT_DIRECTORY ${CMAKE_CURRENT_SOURCE_DIR}/nabia_python_bridge)
endif()
```

SKBUILD is defined only when scikit-build-core runs CMake. Then the two modules and the .py files are installed to nabia_python_bridge/, which becomes that folder inside the wheel. In a normal CMake build (CLion, preset python-bridge) the modules are instead written next to the .py files, for debugging from the source tree.

Static linking. nabia, iir1 and the app's parser are static libraries, and Eigen is header-only, so all of their code is *inside* _core: the wheel needs only the system C++ runtime. A wheel that links a shared library (say libfoo.so) must carry a copy; cibuildwheel's *repair* step (§6) does that.

3.5 Try it

```
python/.venv/bin/python -m pip wheel ./cpp --no-deps -w dist     # one wheel, this Python
MACOSX_DEPLOYMENT_TARGET=13.0 python/.venv/bin/python -m pip wheel ./cpp --no-deps -w dist
uv build --wheel --python 3.12 --out-dir dist cpp                # with uv, another Python
uv build --sdist --out-dir dist cpp                              # the source package
```

4 Installing and using a wheel

```
python -m venv .venv                                    # an isolated environment
.venv/bin/pip install --no-index --find-links dist nabia_python_bridge
```

- -find-links dist adds a folder of wheels as a source; -no-index turns PyPI off, so the package must come from that folder.
- pip picks the matching file, reads its dependencies, installs them, unzips the wheel into .venv/lib/python3.x/site-packages/, and keeps its dist-info there.
- import nabia_python_bridge finds the folder; __init__.py imports _core, and the operating system's loader maps the .so into the process.

Same version, new build. pip skips a package whose version is already installed. After rebuilding the C++ code without changing the version, reinstall explicitly: pip install --force ↪ -reinstall --no-deps .... The setup scripts do that every time:

File 7: python/scripts/setup_env.sh: the wheel, then the project in editable mode

```
#!/usr/bin/env bash
# Sets up python/.venv for the Python project (the interpreter to select in PyCharm):
#   1. installs the nabia_python_bridge wheel built by the C++ project (../dist, see
#      cpp/scripts/build_wheels.sh) - reinstalled every time, so a rebuilt wheel is picked up;
#   2. installs this nabia package in editable mode, with the market data, examples and test extras.
#
#   python/scripts/setup_env.sh                 # uses python3 to create .venv if it does not exist
#   PYTHON=python3.12 python/scripts/setup_env.sh
#   NABIA_WHEEL_DIR=/path/to/wheels python/scripts/setup_env.sh
set -euo pipefail

project="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
wheels="${NABIA_WHEEL_DIR:-$(dirname "$project")/dist}"
venv="$project/.venv"

[[ -x "$venv/bin/python" ]] || "${PYTHON:-python3}" -m venv "$venv"
py="$venv/bin/python"
"$py" -m pip install --quiet --upgrade pip
```

```
# pip picks the wheel matching this interpreter and platform among those in $wheels.
"$py" -m pip install --force-reinstall --no-deps --no-index --find-links "$wheels" nabia_python_bridge
    ↪ || {
    echo "no nabia_python_bridge wheel for $("$py" -V) in $wheels: run cpp/scripts/build_wheels.sh
    ↪ first" >&2
    exit 1; }
"$py" -m pip install --quiet --find-links "$wheels" -e "$project[marketdata,examples,test]"

"$py" -c 'import nabia, nabia_python_bridge as nb; print("nabia", nabia.__file__); print("
    ↪ nabia_python_bridge", nb.__version__, nb.__file__)'
```

Editable installs. `pip install -e python` does not copy `python/nabia`: it installs a pointer to the source folder, so edits are live. That is how `python/` and `webui/` are installed, while the C++ arrives as a finished wheel.

Part II
Many wheels

5 All Python versions on one machine

On macOS and Linux, `build_wheels.sh` loops over versions with uv, which downloads any missing Python:

File 8: `cpp/scripts/build_wheels.sh`

```
#!/usr/bin/env bash
# Builds one nabia_python_bridge wheel per Python version for this machine, and tests each one
# in a clean virtual environment. uv (https://docs.astral.sh/uv/) downloads missing interpreters.
#
#   cpp/scripts/build_wheels.sh                          # Python 3.10 to 3.14
#   cpp/scripts/build_wheels.sh 3.12 3.13                # only these versions
#   NABIA_WHEEL_TEST=0 cpp/scripts/build_wheels.sh       # build without testing
#   NABIA_WHEEL_BUILD_TYPE=RelWithDebInfo cpp/scripts/build_wheels.sh 3.14   # with debug symbols
#
# Wheels go to dist/ at the repository root (NABIA_WHEEL_DIR overrides it), where the Python
# projects install them from (python/scripts/setup_env.sh, webui/scripts/setup_env.sh). The tests
# are the bridge tests of the Python project (python/tests/bridge). On macOS the wheels target
# MACOSX_DEPLOYMENT_TARGET, 13.0 unless already set. Wheels for Linux and Windows are built by
# cibuildwheel (configured in pyproject.toml, run by .github/workflows/wheels.yml).
set -euo pipefail

cpp="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
root="$(dirname "$cpp")"
tests="$root/python/tests/bridge"
build_type="${NABIA_WHEEL_BUILD_TYPE:-Release}"
out="${NABIA_WHEEL_DIR:-$root/dist}"
versions=("$@")
[[ ${#versions[@]} -gt 0 ]] || versions=(3.10 3.11 3.12 3.13 3.14)

command -v uv >/dev/null || { echo "uv is required: https://docs.astral.sh/uv/" >&2; exit 1; }
# Use an installed interpreter (Homebrew, python.org...) before a uv-managed one.
export UV_PYTHON_PREFERENCE="${UV_PYTHON_PREFERENCE:-system}"
[[ "$(uname)" == Darwin ]] && export MACOSX_DEPLOYMENT_TARGET="${MACOSX_DEPLOYMENT_TARGET:-13.0}"

mkdir -p "$out"
work="$(mktemp -d)"
trap 'rm -rf "$work"' EXIT

for v in "${versions[@]}"; do
    echo "=== Python $v ==="
    # Finds the interpreter, downloading it with uv when none is installed.
    py="$(uv python find "$v" 2>/dev/null || { uv python install "$v" >&2 && uv python find "$v"; })"
    # A pre-release (an old uv may only know a 3.x alpha) has no numpy wheels and the wrong ABI.
    "$py" -c 'import sys; sys.exit(sys.version_info.releaselevel != "final")' || {
```

```
        echo "$py is a pre-release Python; update uv (uv self update) or install Python $v" >&2; exit
    ↪ 1; }
    echo "$py ($("$py" -c 'import platform; print(platform.python_version())'))"
    # Isolated build: uv installs scikit-build-core and pybind11 for this interpreter.
    uv build --wheel --python "$py" --out-dir "$work/$v" -C cmake.build-type="$build_type" "$cpp"
    wheel="$(ls "$work/$v"/*.whl)"

    if [[ "${NABIA_WHEEL_TEST:-1}" != 0 ]]; then
        # Install in a fresh environment and run the tests from outside the source tree,
        # so they import the installed package.
        uv venv --quiet --python "$py" "$work/venv-$v"
        uv pip install --quiet --python "$work/venv-$v/bin/python" "$wheel[test]"
        (cd "$work" && "$work/venv-$v/bin/python" -m pytest -q -p no:cacheprovider "$tests")
    fi
    cp "$wheel" "$out/"
done

echo "=== wheels in $out ==="
ls -1 "$out"/*.whl
```

The pattern is the same for every wheel: *build in isolation, install into a clean environment, run the tests against the installed copy, keep it only if they pass.* Testing the installed wheel, from outside the source tree, is the point: it catches a file missing from the wheel, which tests run on the sources would never see.

6 cibuildwheel

cibuildwheel automates exactly that loop, for every platform, and adds what portable wheels need. For each *build identifier* — `cp312-manylinux_x86_64`, `cp313-macosx_arm64`, `cp314-win_amd64`, . . . — it:

1. gets that Python (on Linux, inside a `manylinux` Docker container; on macOS and Windows, official installers it downloads);
2. builds the wheel through the backend, like `pip wheel`;
3. **repairs** it: on Linux `auditwheel` checks it only uses allowed system libraries and copies others in, then sets the `manylinux` tag; on macOS `delocate` does the same for `.dylibs`; on Windows nothing by default;
4. installs it in a fresh environment with `test-extras` and runs `test-command`;
5. moves it to the output folder.

6.1 Configuration

File 9: `cpp/pyproject.toml`: which wheels, and how to test them

```
# Wheels for every supported Python and platform (run by .github/workflows/wheels.yml):
#   pipx run cibuildwheel==4.3.4 --platform linux cpp     (from the repository root; Linux needs Docker)
# Each wheel is tested with the bridge tests of the Python project ({project} is the repository root).
[tool.cibuildwheel]
build = "cp310-* cp311-* cp312-* cp313-* cp314-*"
skip = "*-musllinux_* *-win32 *-manylinux_i686"
test-extras = ["test"]
test-command = "pytest -q -p no:cacheprovider {project}/python/tests/bridge"
```

- `build` and `skip` are glob patterns over identifiers. Here: CPython 3.10–3.14; no musl Linux, no 32-bit.
- `{project}` is the folder cibuildwheel was started in (the repository root, so the Python project's tests are reachable); `{package}` would be `cpp/`.

- Any setting can be overridden by an environment variable: `CIBW_BUILD`, `CIBW_ARCHS`, `CIBW_TEST_SKIP`, ... The workflow and the PowerShell script use that.

File 10: Linux: the container image

```
[tool.cibuildwheel.linux]
# GCC 12+ for C++20; Eigen and iir1 are fetched at configure time.
manylinux-x86_64-image = "manylinux_2_28"
manylinux-aarch64-image = "manylinux_2_28"
```

File 11: macOS: the oldest supported version

```
[tool.cibuildwheel.macos]
environment = { MACOSX_DEPLOYMENT_TARGET = "13.0" }
```

6.2 Running it yourself

```
pipx run cibuildwheel==4.3.4 --print-build-identifiers --platform linux cpp   # what would be built
pipx run cibuildwheel==4.3.4 --platform linux cpp       # Linux wheels, needs Docker running
pipx run cibuildwheel==4.3.4 --platform macos cpp       # on a Mac
CIBW_BUILD="cp312-*" pipx run cibuildwheel==4.3.4 --platform linux cpp   # one version only
```

On Windows, `build_wheels.ps1` wraps the same call: it checks Visual Studio, Git and Python, installs cibuildwheel in a private environment and runs it from the repository root.

File 12: `cpp/scripts/build_wheels.ps1`

```
<#
.SYNOPSIS
    Builds nabia_python_bridge wheels for Windows (win_amd64 by default) with cibuildwheel, and tests
    each one with the bridge tests of the Python project (python/tests/bridge).

.DESCRIPTION
    The Windows counterpart of build_wheels.sh. cibuildwheel downloads every CPython it needs, builds
    the wheel from this C++ project (cpp/pyproject.toml, scikit-build-core, MSVC), installs it in a
    clean environment and runs the tests. The wheels end up in dist\ at the repository root, where
    python\scripts\setup_env.ps1 and webui\scripts\setup_env.ps1 install them from.

    Needs, once:
      - Visual Studio 2022 Build Tools with the "Desktop development with C++" workload (C++20);
      - Git (CMake fetches Eigen and iir1);
      - Python 3.10 or newer with the py launcher (python.org installer), to run cibuildwheel.
    CMake is optional: scikit-build-core downloads it when it is not installed.

.PARAMETER Versions
    CPython versions to build, e.g. 3.12, 3.13. Default: 3.10 to 3.14.

.PARAMETER Arch
    AMD64 (Intel and AMD 64-bit, the default) or ARM64 (on an ARM machine).

.PARAMETER OutDir
    Where the wheels go. Default: dist\ at the repository root.

.PARAMETER NoTest
    Build without running the tests on each wheel.

.PARAMETER PythonExe
    Python used to run cibuildwheel. Default: the py launcher (py -3), else python on the PATH.

.EXAMPLE
    cpp\scripts\build_wheels.ps1
.EXAMPLE
    cpp\scripts\build_wheels.ps1 -Versions 3.12,3.13
.EXAMPLE
    cpp\scripts\build_wheels.ps1 -Versions 3.12 -NoTest
#>
[CmdletBinding()]
```

```powershell
param(
    [string[]] $Versions = @("3.10", "3.11", "3.12", "3.13", "3.14"),
    [ValidateSet("AMD64", "ARM64")] [string] $Arch = "AMD64",
    [string] $OutDir = "",
    [switch] $NoTest,
    [string] $PythonExe = ""
)
$ErrorActionPreference = "Stop"

$cpp = Split-Path -Parent $PSScriptRoot
$root = Split-Path -Parent $cpp
if (-not $OutDir) { $OutDir = Join-Path $root "dist" }
$CibuildwheelVersion = "4.3.4"

# --- Prerequisites -----------------------------------------------------------
$hostArgs = @()
if ($PythonExe) {
    $hostExe = $PythonExe
} elseif (Get-Command py -ErrorAction SilentlyContinue) {
    $hostExe = "py"
    $hostArgs = @("-3")
} elseif (Get-Command python -ErrorAction SilentlyContinue) {
    $hostExe = "python"
} else {
    throw "Python was not found: install Python 3.10+ from python.org."
}
if (-not (Get-Command git -ErrorAction SilentlyContinue)) {
    throw "git was not found: install Git for Windows (CMake fetches Eigen and iir1 with it)."
}
$vswhere = Join-Path ${env:ProgramFiles(x86)} "Microsoft Visual Studio\Installer\vswhere.exe"
if (Test-Path $vswhere) {
    $component = if ($Arch -eq "ARM64") { "Microsoft.VisualStudio.Component.VC.Tools.ARM64" } else { "
    ↪ Microsoft.VisualStudio.Component.VC.Tools.x86.x64" }
    $vs = & $vswhere -latest -products * -requires $component -property installationPath
    if (-not $vs) {
        throw "Visual Studio has no C++ compiler: add the 'Desktop development with C++' workload."
    }
    Write-Host "MSVC: $vs"
} else {
    Write-Warning "vswhere.exe not found; the build fails if Visual Studio 2022 C++ tools are missing."
}

# --- cibuildwheel, in its own virtual environment ----------------------------
$tools = Join-Path ([System.IO.Path]::GetTempPath()) "nabia-cibuildwheel"
if (-not (Test-Path (Join-Path $tools "Scripts\python.exe"))) {
    & $hostExe @hostArgs -m venv $tools
    if ($LASTEXITCODE -ne 0) { throw "could not create $tools" }
}
& (Join-Path $tools "Scripts\python.exe") -m pip install --quiet --upgrade pip "cibuildwheel==
    ↪ $CibuildwheelVersion"
if ($LASTEXITCODE -ne 0) { throw "could not install cibuildwheel" }

# --- Build -------------------------------------------------------------------
# cp312-* etc.; the other settings (skips, tests, test extras) are in cpp\pyproject.toml.
$env:CIBW_BUILD = ($Versions | ForEach-Object { "cp" + ($_ -replace "\.", "") + "-*" }) -join " "
$env:CIBW_ARCHS = $Arch
if ($NoTest) { $env:CIBW_TEST_SKIP = "*" } else { Remove-Item Env:CIBW_TEST_SKIP -ErrorAction
    ↪ SilentlyContinue }

Write-Host "Building $($env:CIBW_BUILD) for $Arch into $OutDir"
# cibuildwheel runs from the repository root: {project} in the test command is the root, so the
# tests of python\tests\bridge are found; "cpp" is the package folder with pyproject.toml.
Push-Location $root
try {
    & (Join-Path $tools "Scripts\cibuildwheel.exe") --platform windows --output-dir $OutDir cpp
    if ($LASTEXITCODE -ne 0) { throw "cibuildwheel failed (exit code $LASTEXITCODE)" }
} finally {
    Pop-Location
}

Write-Host "=== wheels in $OutDir ==="
Get-ChildItem (Join-Path $OutDir "*win*.whl") | ForEach-Object { Write-Host $_.FullName }
```

Part III
GitHub Actions

7 Concepts

GitHub Actions runs commands on machines in GitHub's cloud when something happens in the repository. The vocabulary:

Workflow a YAML file in `.github/workflows/`. GitHub reads every file there. It must be at the repository root (not in `cpp/`).

Event what starts it: a push, a tag, a pull request, a schedule, or a manual start (`workflow_dispatch`). A manual start requires the workflow to exist on the default branch.

Run one execution of a workflow, with a number and a page (Actions tab).

Job a unit of work on *one fresh machine*. Jobs run in parallel unless one `needs` another. Nothing on disk survives from one job to the next.

Runner the machine: `ubuntu-24.04`, `ubuntu-24.04-arm`, `macos-14` (Apple silicon), `windows-latest`, ... A fresh virtual machine each time.

Step one command (`run:`) or one reusable **action** (`uses:`) inside a job, executed in order. If a step fails, the job stops and fails.

Action a packaged step published in a repository, used as `owner/repo@version`: `actions/checkout@v5` clones the repository, `pypa/cibuildwheel@v4.3.4` runs cibuildwheel, `actions/upload-artifact@v4` saves files.

Artifact files a job uploads to the run, to download later or in another job: the way to pass wheels from the build jobs to the collect job, and to you.

Matrix one job definition expanded into several jobs, one per combination of values.

7.1 YAML and expressions

YAML is indentation-based: two spaces per level, `key: value`, lists with `-` , and `{ a: 1, b: 2 }` as a one-line map. Text between `${{` and `}}` is an *expression* GitHub evaluates before running anything:

Expression	Value
`${{ matrix.os }}`	the current matrix value
`${{ github.ref }}`	what triggered the run: `refs/heads/main`, `refs/tags/v0.1.0`
`${{ github.token }}`	a temporary token for this run, with the job's `permissions`
`${{ secrets.NAME }}`	a secret set in Settings → Secrets and variables → Actions
`${{ needs.collect.result }}`	`success`, `failure`, `cancelled` or `skipped`
`startsWith(a, b)`, `!cancelled()`	functions and status checks, used in `if:`

Inside `run:` scripts, the same information is in environment variables (`$GITHUB_REF_NAME`, `$GITHUB_REPOSITORY`, ...).

8 nabia's workflow, piece by piece

8.1 Name and triggers

File 13: `.github/workflows/wheels.yml`: when it runs

```yaml
# Builds and tests nabia_python_bridge wheels for CPython 3.10-3.14 on Linux, macOS and Windows
# with cibuildwheel (settings in cpp/pyproject.toml), testing each with python/tests/bridge. Runs on
# demand, on version tags (v*), and on pull requests that touch the C++ project or the bridge tests.
#
#   wheels          one job per platform; artifact wheels-<os>-<arch>
#   collect         every wheel plus the sdist in one artifact, all-wheels
#   release         on a v* tag: a GitHub Release with all of them attached
#   windows-script  runs cpp/scripts/build_wheels.ps1 and python/scripts/setup_env.ps1 on Windows
name: wheels

on:
  workflow_dispatch:
  push:
    tags: ["v*"]
  pull_request:
    paths: ["cpp/**", "python/tests/bridge/**", "python/examples/0*.py", "python/scripts/**", ".github/
      ↪ workflows/wheels.yml"]
```

- `workflow_dispatch`: the "Run workflow" button and `gh workflow run wheels.yml`.
- `push: tags: ["v*"]`: pushing a tag such as `v0.1.0`. Pushing commits to `main` does *not* start it (no `branches:` filter is given).
- `pull_request: paths:`: only pull requests that change those files.

8.2 The build matrix

File 14: The `wheels` job: one machine per platform

```yaml
  wheels:
    name: ${{ matrix.os }} ${{ matrix.archs }}
    runs-on: ${{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        include:
          - { os: ubuntu-24.04,     archs: x86_64 }
          - { os: ubuntu-24.04-arm, archs: aarch64 }
          - { os: macos-14,         archs: arm64 }
          - { os: macos-14,         archs: x86_64 }   # cross-compiled, tested under Rosetta
          - { os: windows-latest,   archs: AMD64 }    # Windows on Intel/AMD 64-bit (win_amd64)
    steps:
      - uses: actions/checkout@v5
      - uses: pypa/cibuildwheel@v4.3.4
        with:
          package-dir: cpp
          output-dir: wheelhouse
        env:
          CIBW_ARCHS: ${{ matrix.archs }}
      - uses: actions/upload-artifact@v4
        with:
          name: wheels-${{ matrix.os }}-${{ matrix.archs }}
          path: wheelhouse/*.whl
```

- `strategy.matrix.include` lists five combinations, so this definition becomes five jobs running in parallel, each with its own `${{ matrix.os }}` and `${{ matrix.archs }}`.
- `fail-fast: false`: a failure on one platform does not cancel the others.

- `runs-on` picks the machine from the matrix.
- Steps: clone the repository; run cibuildwheel on `cpp` (the action installs it and passes `with:` values as options and `env:` values as `CIBW_*` settings); upload the wheels as an artifact named after the platform. Each runner builds the five Python versions of its platform.
- `macos-14` is an Apple silicon machine; the `x86_64` entry cross-compiles Intel wheels there and tests them under Rosetta.

8.3 Collecting everything

File 15: The `collect` job: every wheel and the sdist in one artifact

```yaml
collect:
  name: all wheels and sdist
  needs: wheels
  if: ${{ !cancelled() }}   # keep the wheels of the platforms that succeeded
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v5
    - uses: actions/download-artifact@v4
      with:
        pattern: wheels-*
        merge-multiple: true
        path: dist
    - name: Source distribution
      run: pipx run build --sdist --outdir dist cpp
    - name: List
      run: ls -l dist
    - uses: actions/upload-artifact@v4
      with:
        name: all-wheels
        path: dist/*
```

- `needs: wheels` waits for all five build jobs.
- By default a job runs only if everything it needs succeeded. `if: ${{ !cancelled() }}` overrides that: it runs even if a platform failed, so the good wheels are not lost.
- `download-artifact` with `pattern` and `merge-multiple` puts the files of all `wheels-*` artifacts into one folder.
- `pipx run build -sdist` builds the source package, the fallback for platforms without a wheel.

8.4 Publishing a release

File 16: The `release` job: a GitHub Release on version tags

```yaml
release:
  name: GitHub Release
  needs: collect
  if: ${{ startsWith(github.ref, 'refs/tags/v') && needs.collect.result == 'success' }}
  runs-on: ubuntu-24.04
  permissions:
    contents: write
  steps:
    - uses: actions/download-artifact@v4
      with:
        name: all-wheels
        path: dist
    - name: Create the release with every wheel attached
      env:
        GH_TOKEN: ${{ github.token }}
```

```
      run: >
        gh release create "$GITHUB_REF_NAME" dist/*
        --repo "$GITHUB_REPOSITORY" --title "nabia $GITHUB_REF_NAME" --generate-notes
```

- The `if:` makes it run only for tags starting with `v`, and only if collecting succeeded. On manual runs it shows as *skipped*.
- `permissions: contents: write`: the run's token is read-only by default; creating a release needs write access, granted to this job only.
- `gh` (the GitHub CLI, installed on every runner) creates the release with all files attached and notes generated from the commits since the previous tag.

8.5 Testing the Windows scripts

File 17: The `windows-script` job

```
windows-script:
  name: Windows build script
  runs-on: windows-latest
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - name: cpp/scripts/build_wheels.ps1 (CPython 3.12, tested)
      shell: pwsh
      run: cpp/scripts/build_wheels.ps1 -Versions 3.12 -PythonExe python
    - name: python/scripts/setup_env.ps1 and the Python project's tests
      shell: pwsh
      run: |
        python/scripts/setup_env.ps1 -PythonExe python
        python/.venv/Scripts/python.exe -m pytest -q python/tests
        if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
```

`actions/setup-python` installs a Python and puts it on the `PATH`; `shell: pwsh` runs the steps with PowerShell. The job builds one wheel with `build_wheels.ps1`, installs it with `setup_env.ps1`, and runs the Python project's tests: the scripts a Windows user would run, run on Windows.

Part IV
Doing it yourself

9 Writing a job

A recipe:

1. **Pick the machine** (`runs-on`), or several with a matrix.
2. **Start with `actions/checkout`**: a runner starts empty.
3. **Install what is missing**: `actions/setup-python` for a given Python; compilers, CMake, Git and `gh` are preinstalled on GitHub's images.
4. **Run commands** with `run:`, one step per logical action, each with a `name:` so the log reads well. Use `working-directory:` instead of `cd`.

5. **Keep results** with `upload-artifact`; get them in another job with `needs:` and `download-artifact`.

6. **Grant only the permissions needed**, per job.

The examples below are not in the workflow; paste them under `jobs:` to use them.

9.1 Example: the C++ tests on Linux and macOS

```yaml
cpp-tests:
  name: C++ tests ${{ matrix.os }}
  strategy:
    fail-fast: false
    matrix:
      os: [ubuntu-24.04, macos-14]
  runs-on: ${{ matrix.os }}
  defaults:
    run:
      working-directory: cpp
  steps:
    - uses: actions/checkout@v5
    - name: Configure
      run: cmake --preset release -DNABIA_BUILD_DOCS=OFF
    - name: Build
      run: cmake --build --preset release -j 4
    - name: Test
      run: ctest --preset release
```

A list under `matrix` (`os: [a, b]`) is the short form of `include`; with several lists, the matrix is their cross product.

9.2 Example: the Python project's tests against the Linux wheel

```yaml
python-tests:
  needs: wheels
  runs-on: ubuntu-24.04
  steps:
    - uses: actions/checkout@v5
    - uses: actions/setup-python@v5
      with:
        python-version: "3.12"
    - uses: actions/download-artifact@v4
      with:
        name: wheels-ubuntu-24.04-x86_64
        path: dist
    - name: Install the wheel and nabia
      run: PYTHON=python python/scripts/setup_env.sh
    - name: Tests
      working-directory: python
      run: .venv/bin/python -m pytest -q
```

9.3 Example: publishing to PyPI without a password

PyPI's *trusted publishing* lets a workflow upload without storing a token: on pypi.org, under the project's Publishing settings, register this repository, the workflow file `wheels.yml` and an environment name (`pypi`). Then:

```yaml
pypi:
  needs: collect
  if: ${{ startsWith(github.ref, 'refs/tags/v') }}
  runs-on: ubuntu-24.04
  environment: pypi              # optional approval rules live in Settings > Environments
  permissions:
    id-token: write              # proves to PyPI which workflow is uploading
```

```
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: all-wheels
          path: dist
      - uses: pypa/gh-action-pypi-publish@release/v1
```

Every upload needs a new version number in `cpp/pyproject.toml`: PyPI never accepts the same version twice. Try it first on test.pypi.org (`with: repository-url: https://test.pypi.org/legacy/`).

9.4 Useful extras

```yaml
concurrency:                        # top level: a new run of the same branch cancels the older one
  group: wheels-${{ github.ref }}
  cancel-in-progress: true

    timeout-minutes: 60             # in a job: stop a hung job early (default 360)

      - uses: actions/cache@v4      # in steps: reuse downloads between runs
        with:
          path: ~/.cache/pip
          key: pip-${{ runner.os }}-${{ hashFiles('cpp/pyproject.toml') }}
```

10 Running, watching and debugging

10.1 From the terminal

Command	Does
`gh workflow run wheels.yml`	starts a run (the file must be on `main`)
`gh run list -workflow wheels.yml`	recent runs with their ids and status
`gh run watch <id>`	live status until it finishes
`gh run view <id>`	jobs and their results
`gh run view <id> -log-failed`	the log lines of failed steps (after the run ends)
`gh run rerun <id> -failed`	reruns only the failed jobs
`gh run cancel <id>`	stops a run
`gh run download <id> -n all-wheels -D dist`	downloads an artifact
`gh release list`, `gh release view v0.1.0`	releases and their files

10.2 On the web

Repository → **Actions** tab → *wheels* in the sidebar → a run. The summary page shows the jobs as a graph (`wheels` → `collect` → `release`) and, at the bottom, the **Artifacts**. Click a job, then a step, to read its log; the gear menu downloads the full log. *Re-run jobs* and *Cancel workflow* are at the top right.

10.3 Reading a failure

cibuildwheel ends a failed build with a summary such as

```
Error: cibuildwheel: Command [... '-m', 'build', '.../cpp', '--wheel', ...] failed with code 1.
```

which only says *that* the build failed. The cause is above it: search the log for `error:` (compiler), `CMake Error`, `FAILED:` (the file that did not compile) or a Python traceback. Failures met while setting up this workflow, and their fixes:

Log	Cause	Fix
`HTTP 404: workflow wheels.yml not found on the default branch`	the file was not pushed to `main`	commit and push it
`AssertionError: uv not found` (macOS, Windows)	`build-frontend = "build[uv]"` needs uv on the runner	use the default frontend
`error: no member named 'jthread' in namespace 'std'`	the Xcode 15 libc++ of `macos-14` is older than the local one	a joining `std::thread` wrapper
`'from_chars' is unavailable: introduced in macOS 26.0`	a deployment target older than the library function	`strtod`/`snprintf`
`The run was canceled by @user`	cancelled by hand	nothing; rerun if needed

Reproduce locally first. A CI round trip takes minutes; most failures reproduce on your machine: `cpp/scripts/build_wheels.sh 3.12` for macOS, `pipx run cibuildwheel --platform linux cpp` with Docker for Linux, an older `MACOSX_DEPLOYMENT_TARGET` for macOS availability errors.

11 Releasing

Where	Who can install	How
Run artifacts	you and collaborators, logged in, for 90 days	`gh run download`
GitHub Release	anyone who can see the repository (it is private: collaborators); permanent	`git tag v0.1.0 && git push origin v0.1.0`, then download from the Releases page
PyPI	anyone: `pip install nabia-python-bridge`	trusted publishing job, new version each time

A release checklist:

1. Set the version in `cpp/pyproject.toml` (and in `python/` and `webui/` if they are published too); commit and push.
2. Start a manual run and check every job is green.
3. Tag and push the tag: `git tag v0.2.0 && git push origin v0.2.0`. The tag run builds again and creates the release.

12 Costs and limits

- **Minutes.** The repository is private, so runs use the account's included Actions minutes, and macOS and Windows minutes count more than Linux ones (see Settings → Billing). A full run builds 25 wheels on five machines plus the Windows script job; trigger it when needed rather than on every push.
- **Storage.** Artifacts count toward the account's storage until they expire (90 days by default); releases do not expire.
- **Versions.** Pin actions to versions (`@v5`, `@v4.3.4`) so a new release of an action cannot change the build unannounced; update them deliberately.

A Glossary

ABI application binary interface: how compiled code calls other compiled code. CPython's changes between minor versions.

auditwheel, delocate, delvewheel tools that copy shared libraries into a wheel and fix its tag (Linux, macOS, Windows).

build backend / frontend the backend (scikit-build-core) turns a project into a wheel; the frontend (pip, `build`, `uv build`) calls it in an isolated environment.

deployment target the oldest macOS a binary supports.

editable install an install that points to the source folder instead of copying it.

glibc the GNU C library of Linux; `manylinux_2_28` means version 2.28 or newer.

sdist source distribution: the sources and `pyproject.toml`, compiled on install.

site-packages the folder of an environment where packages are installed.

wheel tag the `python-abi-platform` part of a wheel's name.